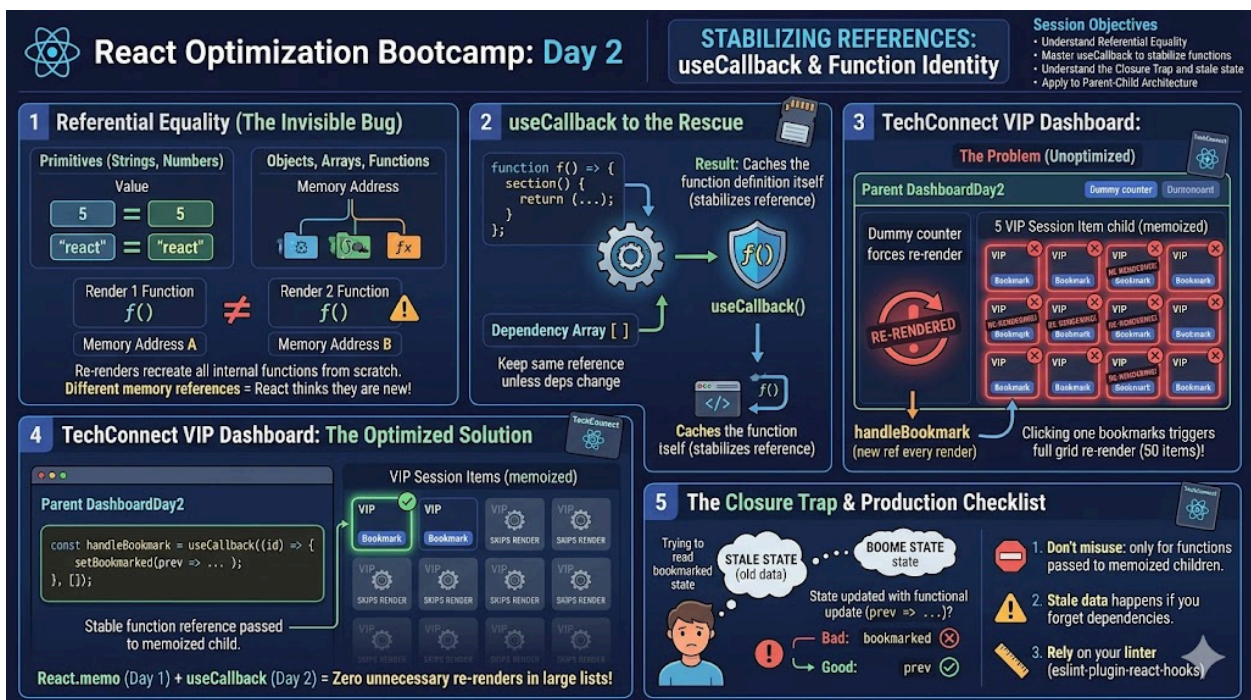


Day 2: The Function Reference Trap (`useCallback`)

Objectives

- Understand **Referential Equality** and why React thinks new functions are created on every render.
- Master `useCallback` to stabilize function references.
- Understand the "Closure Trap" and how stale state can break your app.
- Apply these concepts to a parent-child component architecture.



🧠 Referential Equality (The Invisible Bug)

In JavaScript, primitives (like strings and numbers) are compared by their value. But objects, arrays, and **functions** are compared by their memory address (reference).

Every time a React component re-renders, it recreates all the functions inside of it from scratch.

To React, `Function A (Render 1)` is **NOT EQUAL** to `Function A (Render 2)`.

If you pass a function down to a child component wrapped in `React.memo`, that child will *still* re-render every single time the parent renders, because it thinks it is receiving a brand new function prop!



`useCallback` to the Rescue

`useCallback` is `useMemo`'s sibling. While `useMemo` caches the *result* of a calculation, `useCallback` caches the **actual function definition itself**.

By wrapping a function in `useCallback`, you tell React: *"Keep the exact same memory reference for this function unless the variables in my dependency array change."*



TechConnect VIP Dashboard

Today, we built the VIP Dashboard for **TechConnect**.

The Problem: We had a list of VIP sessions. We wanted to bookmark one session. But because the parent `DashboardDay2` component held the `handleBookmark` function, clicking *one* bookmark button forced all 50 sessions to completely re-render, causing the UI to stutter.

The Optimized Solution

We wrapped our `SessionItem` in `React.memo` (from Day 1), and wrapped the `handleBookmark` function in `useCallback` so the child components never receive a new function reference unless absolutely necessary.

JavaScript

```
import React, { useState, useCallback, memo } from 'react';

// Simulated data: 50 VIP Sessions
const sessions = Array.from({ length: 50 }, (_, i) => ({
  id: i,
  name: `Masterclass: React Architecture ${i}`,
}));

export default function DashboardDay2() {
  const [bookmarked, setBookmarked] = useState([]);
  const [counter, setCounter] = useState(0); // Dummy state to trigger parent re-renders
```

```

// OPTIMIZATION: useCallback
// We use the functional state update (prev => ...) inside setBookmarked to avoid adding
// 'bookmarked' to the dependency array, preventing the "Closure Trap"!
const handleBookmark = useCallback((id) => {
  setBookmarked((prev) =>
    prev.includes(id) ? prev.filter(b => b !== id) : [...prev, id]
  );
}, []);

return (
  <div className="p-8 bg-white rounded-2xl shadow-sm border border-slate-200">
    <div className="flex justify-between items-center mb-6">
      <h2 className="text-2xl font-bold text-slate-900">VIP Dashboard (Day 2)</h2>
      <button
        onClick={() => setCounter(c => c + 1)}
        className="px-4 py-2 bg-emerald-100 text-emerald-700 font-semibold rounded-md
        hover:bg-emerald-200"
      >
        Force Parent Render ({counter})
      </button>
    </div>

    <p className="mb-4 text-slate-600 font-medium">Bookmarked Sessions: {bookmarked.length}</p>

    <div className="grid grid-cols-1 md:grid-cols-2 gap-4">
      {sessions.map(session => (
        <SessionItem
          key={session.id}
          session={session}
          isBookmarked={bookmarked.includes(session.id)}
          onBookmark={handleBookmark}
        />
      ))}
    </div>
  </div>
);
}

// OPTIMIZATION: React.memo
// This child now safely ignores parent re-renders because 'onBookmark' is referentially
// stable!
const SessionItem = memo(({ session, isBookmarked, onBookmark }) => {
  // Artificial delay to make unoptimized re-renders visible in the UI
  let startTime = performance.now();
  while (performance.now() - startTime < 5) {}

  console.log(`Rendering Session ${session.id}... 🕒`);

  return (
    <div className="p-4 border border-slate-200 rounded-lg flex justify-between items-center

```

```

bg-slate-50">
  <span className="font-medium text-slate-800">{session.name}</span>
  <button
    onClick={() => onBookmark(session.id)}
    className={`px-4 py-2 rounded-md text-sm font-bold transition-colors ${
      isBookmarked ? 'bg-indigo-600 text-white' : 'bg-slate-200 text-slate-700 hover:bg-
    slate-300'
    }`}
  >
    {isBookmarked ? 'Bookmarked' : 'Bookmark'}
  </button>
</div>
);
});

```

Production & Interview Checklist

- **Don't `useCallback` everything:** If you are passing a function directly to a native DOM element like a `<button onClick={handleClick}>`, `useCallback` does absolutely nothing for performance. Only use it when passing functions to custom child components wrapped in `React.memo`.
- **The Closure Trap:** If your `useCallback` function relies on a state variable, but you forget to put that variable in the `[]` dependency array, the function will execute using "stale" (old) data. Always rely on your linter (`eslint-plugin-react-hooks`) to catch this!